

Хід виконання:

Завдання:

Налаштувати систему виявлення атак (SNORT або Suricata), протестувати її роботу на наявних в комплекті сигнатурах трафіку та додати нову сигнатуру, що описує особливості деяких пакетів відповідно індивідуального завдання. При цьому треба якомога точніше враховувати особливості протоколу та контексту передачі даних (іншими словами, ідентифкація протоколу по номерам портів не буде достатньою). Для відторення необхідного трафіка можливо використовувати утиліту `tcpreplay`

Спочатку я налаштувала мережеві інтерфейси на Kali1, яка буде виступати в ролі IPS-шлюзу. Використовуючи команду `ip addr add`, призначила інтерфейсу `eth0` адресу `192.168.1.1` для зв'язку із зовнішньою мережею, а інтерфейс `eth1` активувала та налаштувала на адресу `192.168.2.1` для внутрішнього сегмента:

```
(kali@kali)-[~]
$ sudo ip addr add 192.168.1.1/24 dev eth0

(kali@kali)-[~]
$ sudo ip link set eth1 up

(kali@kali)-[~]
$ sudo ip addr add 192.168.2.1/24 dev eth1
```

Перевірка конфігурації за допомогою `ip a`:

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:7a:cc:e9 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.1/24 scope global eth0
       valid_lft forever preferred_lft forever
5: tun0: <POINTOPOINT,NOARP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
   link/none
   inet 10.8.0.1/24 brd 10.8.0.255 scope global tun0
       valid_lft forever preferred_lft forever
6: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:7a:cc:f3 brd ff:ff:ff:ff:ff:ff
   inet 192.168.2.1/24 scope global eth1
       valid_lft forever preferred_lft forever
   inet6 fe80::20c:29ff:fe7a:ccf3/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Бачимо що обидва мережеві адаптери працюють коректно.

Дозволяємо пересилання пакетів між інтерфейсами:

```
(kali㉿kali)-[~]  
$ sudo sysctl -w net.ipv4.ip_forward=1  
net.ipv4.ip_forward = 1
```

Тепер трафік зможе вільно проходити через наш майбутній фільтр Snort.

Далі на другій віртуальній машині, яка імітує зовнішній хост для тестів, налаштувала IP-адресу 192.168.1.10 на інтерфейсі eth0. Також через команду ip route встановила першу Kali як шлюз за замовчуванням:

```
(ilonka㉿kali)-[~]  
$ sudo ip addr add 192.168.1.10/24 dev eth0  
[sudo] password for ilonka:  
  
(ilonka㉿kali)-[~]  
$ sudo ip route add default via 192.168.1.1  
  
(ilonka㉿kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:98:4e:c6 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.1.10/24 scope global eth0  
        valid_lft forever preferred_lft forever  
    inet6 fe80::20c:29ff:fe98:4ec6/64 scope link proto kernel_ll  
        valid_lft forever preferred_lft forever
```

Виконала перевірку через ip a та бачу, що параметри застосувалися правильно.

Перевіряю досяжність між хостами:

```
(ilonka㉿kali)-[~]  
$ ping 192.168.1.1  
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.  
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=10.4 ms  
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.833 ms  
^C  
— 192.168.1.1 ping statistics —  
2 packets transmitted, 2 received, 0% packet loss, time 1002ms  
rtt min/avg/max/mdev = 0.833/5.624/10.415/4.791 ms
```

Все працює коректно, пінг від калі2 до калі1 проходить.

```

kali@kali:~$ sudo apt update
sudo apt install snort -y
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [23.1 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/contrib Sources [83.9 kB]
Get:5 http://kali.download/kali kali-rolling/main Sources [17.6 MB]
Get:6 http://kali.download/kali kali-rolling/non-free Sources [115 kB]
Get:7 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:8 http://kali.download/kali kali-rolling/main i386 Packages [20.4 MB]
Get:9 http://kali.download/kali kali-rolling/main i386 Contents (deb) [47.6 MB]
Get:10 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.7 MB]
Get:11 http://kali.download/kali kali-rolling/contrib i386 Packages [100 kB]
Get:12 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:13 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [274 kB]
Get:14 http://kali.download/kali kali-rolling/contrib i386 Contents (deb) [191 kB]
Get:15 http://kali.download/kali kali-rolling/non-free i386 Packages [140 kB]
Get:16 http://kali.download/kali kali-rolling/non-free amd64 Packages [184 kB]
Get:17 http://kali.download/kali kali-rolling/non-free i386 Contents (deb) [846 kB]
Get:18 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [880 kB]
Get:19 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [14.3 kB]
Get:20 http://kali.download/kali kali-rolling/non-free-firmware i386 Packages [13.8 kB]
Get:21 http://kali.download/kali kali-rolling/non-free-firmware i386 Contents (deb) [33.8 kB]
Get:22 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [33.8 kB]
Fetched 162 MB in 34s (4,712 kB/s)
2310 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
  libavformat60 libnhttp3-3 libpostal1 libswscale7 libwiretap14t64 libxatracker2
  liblua5.2-0 libpostal-data libssh-gcrypt-4 libwireshark17t64 libwsutil15t64
Use 'sudo apt autoremove' to remove them.

Upgrading:
  libhwloc-plugins libhwloc15 libze1

Installing:
  snort

Installing dependencies:
  libdaq3 libestr0 libfastjson4 liblognorm5 oinkmaster rsyslog snort-common snort-common-libraries snort-rules-default

```

```
(kali㉿kali)-[~]  
$ snort -V  
  
,,_~  
o" )~  
''''  
- * > Snort++ < * -  
Version 3.11.1.0  
By Martin Roesch & The Snort Team  
http://snort.org/contact#team  
Copyright (C) 2014-2026 Cisco and/or its affiliates. All rights reserved.  
Copyright (C) 1998-2013 Sourcefire, Inc., et al.  
Using DAQ version 3.0.24  
Using libpcap version 1.10.5 (with TPACKET_V3)  
Using LuaJIT version 2.1.1761786044  
Using LZMA version 5.8.1  
Using OpenSSL 3.5.4 30 Sep 2025  
Using PCRE2 version 10.46 2025-08-27  
Using ZLIB version 1.3.1
```

```

(kali@kali)~$ sudo snort -c /etc/snort/snort.lua -i eth0
[sudo] password for kali:

o")~ Snort++ 3.11.1.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
  active
  alerts
  daq
  decode
  host_cache
  hosts
  packets
  socks
  ssh
  ssl
  telnet
  cip
  dnp3
  iec104
  mms
  modbus
  opcua
  dce_smb
  dce_tcp
  dce_http_server
  gtp_inspect
  port_scan
  smtp
  ftp_server
  output
  ftp_client
  ftp_data
  http_inspect
  http2_inspect
  file_policy
  js_norm
  wizard
  binder
  pop

```

```

Finished /etc/snort/snort.lua:
Loading file_id.rules_file:
Loading file_magic.rules:
Finished file_magic.rules:
Finished file_id.rules_file:

ips policies rule stats
  id loaded shared enabled file
  0 219 0 219 /etc/snort/snort.lua

rule counts
  total rules loaded: 219
  text rules: 219
  option chains: 219
  chain headers: 1

service rule counts
  file_id: 219 219
  total: 219 219

fast pattern groups
  to_server: 1
  to_client: 1

search engine (ac_bnfa)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 3072
appid: patterns loaded: 300

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] eth0

```

З логів бачу успішне завантаження всіх необхідних модулів інспекції та

базових правил безпеки. Система проаналізувала конфігурацію, завантажила 219 текстових правил і перейшла у стан обробки пакетів у реальному часі ("Commencing packet processing").

Далі я створила власне правило для Snort у файлі local.rules, яке налаштоване на виявлення будь-яких ICMP-пакетів (пінгів) у мережі. Кожна така подія супроводжується повідомленням "ALARM: ICMP Ping Detected", та має свій унікальний ідентифікатор:

```
File Actions Edit View Help
GNU nano 8.7 /etc/snort/local.rules
$Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $ /rules/local.rules ping_hostA.pcap
# _____
# LOCAL RULES
# _____
# This file intentionally does not come with signatures. Put your local
# additions here.
alert icmp any any → any any (msg:"ALARM: ICMP Ping Detected"; sid:1000001; rev:1;)
```

Після цього я запустила Snort 3, вказавши шлях до основного конфігураційного файлу та свого нового правила.

```
(kali@kali)~[~]
$ sudo snort -c /etc/snort/snort.lua -i eth0 -R /etc/snort/local.rules -A alert_fast -k none

o")~ Snort++ 3.11.1.0

Loading /etc/snort/snort.lua: OK
Loading snort_defaults.lua: OK
Finished snort_defaults.lua: OK
Loading rule args: OK
Loading /etc/snort/local.rules: OK
Finished rule args: OK

ips policies rule stats
id loaded shared enabled file
0 0 220 0 220 /etc/snort/snort.lua

rule counts
total rules loaded: 220
text rules: 220
option chains: 220
chain headers: 2
```

Можемо помітити що правил стало на 1 більше, тобто 220, а не 219 як раніше.

Тепер перевіримо чи були спрацювання у конкретному файлі:

```
(kali@kali)-[~/bkm/lab5]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r ping_hostA.pcap -A alert_talos
[sudo] password for kali:
##### ping_hostA.pcap #####
[1:1000001:1] ALARM: ICMP Ping Detected (alerts: 3)
#####
```

Бачимо що дійсно були помічені пакети протоколи ICMP.

Індивідуальні завдання:

Завдання 1: Виявлення повідомлень HTTP base та digest authentication

Заходжу у файл локальних правил та пишу правило шукати в цьому потоці даних фрази: "Authorization: Basic" та "Authorization: Digest":

```
GNU nano 8.7 local.rules
# Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures. Put your local
# additions here.
alert tcp any any -> any 80 (msg:"HTTP Basic Auth detected"; content:"Authorization: Basic"; sid:1000002; rev:1;)
alert tcp any any -> any 80 (msg:"HTTP Digest Auth detected"; content:"Authorization: Digest"; sid:1000003; rev:1;)
```

Це дозволить нам бачити, хто і коли намагається авторизуватися в мережі.

На Kali2 використовуємо команду curl, ми звертаємося до адреси 192.168.1.1 і відправляємо туди логін user та пароль password. Тобто ми просто створюємо реальний мережевий запит:

```
(ilonka@kali)-[~]
$ curl -v http://192.168.1.1 --user user:password
* Trying 192.168.1.1:80 ...
* Established connection to 192.168.1.1 (192.168.1.1 port 80) from 192.168.1.10 port 56728
* using HTTP/1.x
* Server auth using Basic with user 'user'
> GET / HTTP/1.1
> Host: 192.168.1.1
> Authorization: Basic dXNlcjpwYXNzd29yZA==
> User-Agent: curl/8.17.0
> Accept: */*
```

Бачу, що запит пройшов успішно: термінал показує рядок Authorization: Basic ..., де наші дані (тобто логін і пароль) передаються у закодованому вигляді.

Далі запускаю Snort, вказуючи йому використовувати наші створені правила (local.rules) та даємо йому файл test1.pcap, де збережений наш попередній запит з логіном та паролем:

```
(kali@kali)-[~/bkm/lab5]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r test1.pcap -A alert_talos
##### test1.pcap #####
[1:1000002:1] HTTP Basic Auth detected (alerts: 2)
#####
```

Бачимо що Snort розпізнав передачу пароля в потоці даних і видав попередження. Ми отримали два сповіщення.

Таким чином також шукаю всі рядки, де є Authorization:

```
(kali@kali)-[~/bkm/lab5]
$ tcpdump -r test1.pcap -A | grep Authorization
reading from file test1.pcap, link-type EN10MB (Ethernet), snapshot length 262144
Authorization: Basic dXNlcjpwYXNzd29yZA==
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

Тобто фактично знаходжу логіни і паролі в HTTP-трафіку задля перевірки самого створеного рсар файлу.

Завдання 2:

У перехопленні HTTP трафіку знайдіть усі Cookie та збережіть їх у файл.

Створюю нове правило як тільки в будь-якому HTTP-запиті програма бачить заголовок з текстом "Cookie", вона створює сповіщення з повідомленням "HTTP Cookie Found":

```
GNU nano 8.7 local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert http any any -> any any (msg:"HTTP Cookie Found"; http_raw_header; content:"Cookie"; sid:1000006; rev:5;)
```

Таким чином зможемо знайти та виділити куки з усього потоку даних, що проходять через мережу.

З калі2 до калі1 передаю два значення: SessionID (унікальний ключ сесії) та User (ім'я користувача):

```
(ilonka@kali)-[~]
$ curl -v -b "SessionID=Kali-Perfect-Cookie-18744; User=Ilonka2" http://192.168.1.1
* Trying 192.168.1.1:80 ...
* Established connection to 192.168.1.1 (192.168.1.1 port 80) from 192.168.1.10 port 34592
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.1.1
> User-Agent: curl/8.17.0
> Accept: */*
> Cookie: SessionID=Kali-Perfect-Cookie-18744; User=Ilonka2
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 18 Mar 2026 15:27:21 GMT
< Server: Apache/2.4.66 (Debian)
< Last-Modified: Sun, 18 Aug 2024 19:55:22 GMT
< ETag: "29cf-61ffa9299a5e0"
< Accept-Ranges: bytes
< Content-Length: 10703
```

Бачу що сервер прийняв цей запит і відповів кодом 200 OK, що означає успішне з'єднання.

Далі я запустила перевірку раніше записаного файлу з мережевим трафіком, використовуючи те саме створене правило:

```
(kali㉿kali)-[~/bkm/lab5]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r http.pcap -A alert_talos

##### http.pcap #####
[1:1000006:5] HTTP Cookie Found (alerts: 4)
#####
local.rules
```

Snort успішно спіймав чотири випадки передачі куків у мережі.

Також можемо подивитись детально вміст лог-файлу alert, який створив Snort:

```
(kali㉿kali)-[~/bkm/lab5]
$ sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r http.pcap -d -A alert_fast -k none > final_report.txt
(kali㉿kali)-[~/bkm/lab5]
$ cat final_report.txt

o")~ Snort++ 3.11.1.0
Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
references
classifications
ips
ssl
wizard
telnet
cip
dnp3
file_id
modbus
opcua
s7commplus
ftp_client
ftp_server
smtp

Retry queue interval is: 200 ms
++ [0] http.pcap uses
03/18-11:26:35.011468 [**] [1:1000006:5] "HTTP Cookie Found" [**] [Priority: 0] {TCP} 192.168.1.10:34088 → 192.168.1.1:80

http_inspect.http_method[3]:
-----
47 45 54 /etc/snort/rules GET
-----

http_inspect.http_version[8]:
-----
48 54 54 50 2F 31 2E 31 HTTP/1.1
-----

http_inspect.http_uri[1]:
-----
2F /
-----

http_inspect.http_header[57]:
-----
48 6F 73 74 3A 20 31 39 32 2E 31 36 38 2E 31 2E Host: 192.168.1.1
31 0D 0A 55 73 65 72 2D 41 67 65 6E 74 3A 20 63 1..User-Agent: curl/8.17.0
75 72 6C 2F 38 2E 31 37 2E 30 0D 0A 41 63 63 65 url/8.17.0
70 74 3A 20 2A 2F 2A 0D 0A pt: */*.
-----

http_inspect.http_cookie[48]:
-----
53 65 73 73 69 6F 6E 49 44 3D 4B 61 6C 69 2D 50 SessionID=Kali-P
65 72 66 65 63 74 2D 43 6F 6F 68 69 65 2D 31 32 erfect-Cookie=12
33 34 35 38 20 55 73 65 72 3D 49 6C 6F 6E 68 61 345; User=Ilonka
-----

03/18-11:26:42.513062 [**] [1:1000006:5] "HTTP Cookie Found" [**] [Priority: 0] {TCP} 192.168.1.10:37462 → 192.168.1.1:80

http_inspect.http_method[3]:
-----
47 45 54 GET
-----

http_inspect.http_version[8]:
-----
48 54 54 50 2F 31 2E 31 HTTP/1.1
-----

http_inspect.http_uri[1]:
-----
```

Та бачимо самі куки, які ми шукали з детальною інформацією.

Наступне завдання 3:

Виявити пакети HTTP, в яких передаються файли (зберегти дані пакетів, якщо у використуваній версії програмного забезпечення буде така можливість)

Пишу нове правило. В мережевому трафіку слово "Content-Disposition" це мітка в HTTP-запитах, яка зазвичай з'являється тоді, коли сервер або браузер передають якийсь файл. Як тільки система бачить цю мітку, вона має видати сповіщення з відповідним текстом:

```
GNU nano 8.7 local.rules
$Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert http any 80 -> any any (msg:"HTTP File Download Detected"; http_header; content:"Content-Type: application/zip"; file_data; content:"PK", depth 2; sid:1000008; rev:1;)
```

Таким чином ми зможемо відстежувати моменти, коли хтось завантажує або надсилає файли в мережі.

Створюю файл test.txt із змістом та за допомогою утиліти zip цей файл запакуюю в архів. Після створення переміщую архів до кореневої директорії вебсервера Apache, щоб змоделювати завантаження файлу через протокол HTTP:

```
(kali@kali)-[~/bkm/lab5]
$ echo "file for 5 lab" > test.txt

(kali@kali)-[~/bkm/lab5]
$ zip secret.zip test.txt
adding: test.txt (stored 0%)

(kali@kali)-[~/bkm/lab5]
$ sudo mv secret.zip /var/www/html/
```

Саме його пізніше буду завантажувати на Калі2.

За допомогою утиліти tcpdump ініціюю процес захоплення мережевого трафіку на інтерфейсі eth0. Фільтрацію здійснюю за портом 80 (HTTP) для фіксації активності вебсервера під час звернення до нього іншого вузла мережі:

```
(kali@kali)-[~/bkm/lab5]
$ sudo tcpdump -i eth0 port 80 -w file1.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C30 packets captured
30 packets received by filter
0 packets dropped by kernel
```

Бачимо що усі перехоплені пакети, загальна кількість яких склала 30 одиниць, були збережені у файл дампа file1.pcap.

```
(ilonka@kali)-[~]
$ wget --no-cache --no-cookies http://192.168.1.1/secret.zip
--2026-03-18 11:47:39-- http://192.168.1.1/secret.zip
Connecting to 192.168.1.1:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 181 [application/zip]
Saving to: 'secret.zip.2'

secret.zip.2          100%[=====] 181 --.-KB/s  in 0s

2026-03-18 11:47:39 (14.0 MB/s) - 'secret.zip.2' saved [181/181]

(ilonka@kali)-[~]
$ wget --no-cache --no-cookies http://192.168.1.1/secret.zip
--2026-03-18 11:47:41-- http://192.168.1.1/secret.zip
Connecting to 192.168.1.1:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 181 [application/zip]
Saving to: 'secret.zip.3'

secret.zip.3          100%[=====] 181 --.-KB/s  in 0s

2026-03-18 11:47:41 (14.5 MB/s) - 'secret.zip.3' saved [181/181]
```

Запускаю Snort в режимі читання створеного раніше дампа трафіку file1.pcap з врахуванням правила:

```
(kali㉿kali)-[~/bkm/lab5]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r file1.pcap -A alert_talos
local.rules
#### file1.pcap ####
[1:1000008:1] HTTP File Download Detected (alerts: 3)
local.rules
####
```

Також можемо переглянути повний лог з детальною інформацією:

```
(kali㉿ kali)-[~/bkm/lab5]
$ sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r file1.pcap -X -d -k none > file1.txt

(kali㉿ kali)-[~/bkm/lab5]
$ cat file1.txt

o")~   Snort++ 3.11.1.0

Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
    trace /etc/snort/rules
    ips
    references community-games.rules community-sip.rules community-web-nisc.rules
    binder community-trap.rules community-ntp.rules community-web-php.rules
    wizard community-tftp.rules community-sql-injection.rules misc.rules
    appid community-inappropriate.rules community-virus.rules doberman.rules
    js_norm community-mail-client.rules community-web-attacks.rules misc.rules
    file_policy community-nisc.rules community-web-fuzz.rules dos.rules
    file_id community-ntp.rules community-web-client.rules experimental.rules
    http2_inspect community-practice.rules community-web-dos.rules exploit.rules
    http_inspect community-policy.rules community-web-tls.rules finger.rules
    ftp_data
    ftp_client /etc/snort/rules
    ftp_server
```

File Statistics					
file stats					
	Type	TypeDn	TypeUp	BytesDn	BytesUp
	ZIP(29)	3	0	0	0
	Total	3	0	0	0

Тут також можемо помітити що передавались 3 архіви.

Завдання 4:

Виявлення завантаження ч-з HTTP файлів певних форматів

Створюю правила, що дозволяють виявляти завантаження файлів через HTTP певних форматів. Налаштовую сповіщення про завантаження файлів типу EXE, PDF та PNG. Для цього використовую сигнатури файлів (байти на початку файлу), які Snort перевіряє у HTTP-трафіку:

```
GNU nano 8.7 local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert http any any -> any any (msg:"HTTP EXE Download Detected"; file_data; content:"MZ", depth 2; sid:1000009; rev:2;)
alert http any any -> any any (msg:"HTTP PDF Download Detected"; file_data; content:"%PDF", depth 4; sid:1000010; rev:2;)
alert http any any -> any any (msg:"HTTP PNG Image Detected"; file_data; content:"\89 50 4E 47 0D 0A 1A 0A", depth 8; sid:1000011; rev:2;)
```

Якщо система знаходить відповідну сигнатуру, вона генерує повідомлення (alert) про виявлене завантаження.

Далі створюю тестові файли різних форматів за допомогою команди echo. У файли записую сигнатури (перші байти), які відповідають цим типам файлів. Після цього створені файли переміщую у каталог /var/www/html/, щоб їх можна було завантажити через HTTP:

```
(kali@kali)-[~/bkm/lab5]
$ echo "MZtest" > test.exe
(kali@kali)-[~/bkm/lab5]
$ echo "%PDF-1.5" > test.pdf
(kali@kali)-[~/bkm/lab5]
$ echo -e "\x89PNG\r\n\x1a\n" > test.png
(kali@kali)-[~/bkm/lab5]
$ sudo mv test.exe test.pdf test.png /var/www/html/
```

У висновку маємо 3 створені файли, які згодом будемо завантажувати.

Далі запускаю tcpdump для перехоплення мережевого трафіку на інтерфейсі eth0 по порту 80 (HTTP). Захоплений трафік записую у файл down1.pcap:

```
(kali㉿kali)-[~/bkm/lab5]
$ sudo tcpdump -i eth0 port 80 -w down1.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C30 packets captured
30 packets received by filter
0 packets dropped by kernel
```

Таким чином зафіксуємо процес завантаження тестових файлів через HTTP і пізніше проаналізуємо його.

Виконую завантаження цих трьох файлів з Калі2:

```
(ilonka㉿kali)-[~]
$ wget http://192.168.1.1/test.exe
--2026-03-18 12:05:37-- http://192.168.1.1/test.exe
Connecting to 192.168.1.1:80 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7 [application/x-msdos-program]
Saving to: 'test.exe.1'

test.exe.1          100%[=====>]          7  --.-KB/s   in 0s

2026-03-18 12:05:37 (494 KB/s) - 'test.exe.1' saved [7/7]

(ilonka㉿kali)-[~]
$ wget http://192.168.1.1/test.pdf
--2026-03-18 12:05:41-- http://192.168.1.1/test.pdf
Connecting to 192.168.1.1:80 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9 [application/pdf]
Saving to: 'test.pdf.1'

test.pdf.1          100%[=====>]          9  --.-KB/s   in 0s

2026-03-18 12:05:41 (570 KB/s) - 'test.pdf.1' saved [9/9]

(ilonka㉿kali)-[~]
$ wget http://192.168.1.1/test.png
--2026-03-18 12:05:44-- http://192.168.1.1/test.png
Connecting to 192.168.1.1:80 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9 [image/png]
Saving to: 'test.png.1'

test.png.1          100%[=====>]          9  --.-KB/s   in 0s

2026-03-18 12:05:44 (407 KB/s) - 'test.png.1' saved [9/9]
```

Всі файли було успішно завантажено.

Далі я запускаю Snort і перевіряю мережевий трафік з файлу:

```
(kali㉿kali)-[~/bkm/lab5]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r down1.pcap -A alert_talos

##### down1.pcap ##### /rules
[1:1000009:2] HTTP EXE Download Detected (alerts: 1)
[1:1000010:2] HTTP PDF Download Detected (alerts: 1)
[1:1000011:2] HTTP PNG Image Detected (alerts: 1)
##### local.rules
```

Snort аналізує дані та показує, що було виявлено завантаження файлів через HTTP різних типів: .exe, .pdf та .png. Тобто система успішно визначила підозрілі завантаження.

Також запускаю Snort з іншим форматом виводу (детальнішим).

```
(kali@kali)~[~/bkm/lab5]
$ sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r down1.pcap -q -A alert_fast
03/18-12:05:37.007114 [**] [1:1000009:2] "HTTP EXE Download Detected" [**] [Priority: 0] {TCP} 192.168.1.1:80 → 192.168.1.10:45696
03/18-12:05:40.328587 [**] [1:1000010:2] "HTTP PDF Download Detected" [**] [Priority: 0] {TCP} 192.168.1.1:80 → 192.168.1.10:45710
03/18-12:05:43.898916 [**] [1:1000011:2] "HTTP PNG Image Detected" [**] [Priority: 0] {TCP} 192.168.1.1:80 → 192.168.1.10:48060
```

У результаті бачимо точний час події, IP-адреси джерела і отримувача, а також тип файлу, який завантажувався через HTTP.

Завдання 5:

Виявлення повідомлень SNMP з деякою строкою Community

Я створюю правило що якщо в мережевому трафіку зустрічається рядок admin123, Snort згенерує сповіщення:

```
GNU nano 8.7 local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures. Put your local
# additions here.
alert udp any any → any 161 (msg:"SNMP Community String 'admin123' Detected"; content:"admin123"; sid:1000012; rev:1;)
```

Це дозволяє знаходити підозрілі або ж небезпечні SNMP-запити.

Далі я оновлюю список пакетів та встановлюю утиліту SNMP на Kali Linux за допомогою команди apt:

```
(kali@kali)~[~/bkm/lab5]
$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
Get:2 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:3 https://packages.microsoft.com/repos/code stable/main amd64 Packages [23.4 kB]
Fetched 27.0 kB in 1s (34.3 kB/s)
2326 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  snmp
Installing dependencies:
  libsnmp45
Summary:
  Upgrading: 1, Installing: 1, Removing: 0, Not Upgrading: 2325
  Download size: 2,814 kB
  Space needed: 5,599 kB / 11.0 GB available
Get:1 http://http.kali.org/kali kali-rolling/main amd64 libsnmp45 amd64 5.9.5.2+dfsg-2.1 [2,635 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 snmp amd64 5.9.5.2+dfsg-2.1 [179 kB]
Fetched 2,814 kB in 1s (2,354 kB/s)
Selecting previously unselected package libsnmp45:amd64.
(Reading database ... 585625 files and directories currently installed.)
Preparing to unpack .../libsnmp45_5.9.5.2+dfsg-2.1_amd64.deb ...
Unpacking libsnmp45:amd64 (5.9.5.2+dfsg-2.1) ...
Preparing to unpack .../snmp_5.9.5.2+dfsg-2.1_amd64.deb ...
Unpacking snmp (5.9.5.2+dfsg-2.1) over (5.9.4+dfsg-2) ...
Setting up libsnmp45:amd64 (5.9.5.2+dfsg-2.1) ...
Setting up snmp (5.9.5.2+dfsg-2.1) ...
Processing triggers for kali-menu (2025.4.3) ...
Processing triggers for libc-bin (2.41-12) ...
Processing triggers for man-db (2.13.1-1) ...
```

Щоб мати можливість відправляти SNMP-запити та тестувати роботу правил Snort.

Далі я запускаю tcpdump, щоб перехоплювати SNMP-трафік (порт 161) і записую його у файл snmp.pcap:

```
(kali@kali)-[/etc/snort/rules]
$ sudo tcpdump -i eth0 udp port 161 -w snmp.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C6 packets captured
6 packets received by filter
0 packets dropped by kernel
```

У результаті бачимо, що було захоплено 6 пакетів - тобто SNMP-трафік успішно зібрано для подальшого аналізу.

На калі2 пробує відправити SNMP-запит командою snmpget, використовуючи community string admin123:

```
(ilonka@kali)-[~]
$ snmpget -v 2c -c admin123 192.168.1.1 1.3.6.1.2.1.1.1.0
Timeout: No Response from 192.168.1.1.
```

Отримую відповідь Timeout, тобто пристрій не відповів, але сам запит все одно був відправлений у мережу.

Далі запускаю Snort і аналізую збережений файл snmp.pcap:

```
(kali@kali)-[/etc/snort/rules]
$ sudo snort -q -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r snmp.pcap -A alert_talos

##### snmp.pcap #####
[1:1000012:1] SNMP Community String 'admin123' Detected (alerts: 6)
#####
```

Snort знаходить пакети, де є рядок admin123, і показує сповіщення про виявлення цього community string.

Переглядаю у більш детальному вигляді те саме:

```
(kali@kali)-[/etc/snort/rules]
$ sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -r snmp.pcap -q -A alert_fast
03/18-12:11:24.276196 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
03/18-12:11:25.278520 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
03/18-12:11:26.279958 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
03/18-12:11:27.281586 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
03/18-12:11:28.283103 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
03/18-12:11:29.285580 [**] [1:1000012:1] "SNMP Community String 'admin123' Detected" [**] [Priority: 0] {UDP} 192.168.1.10:41436 → 192.168.1.1:161
```

Бачимо точний час кожного пакета, IP-адреси (звідки і куди йшов запит), що це UDP-трафік на порт 161 і що в кожному пакеті є admin123.

Завдання 6:

Виявлення в потоці даних SSL сертифікатів з деякою ЦП (по фрагменту певної довжини)

Я пишу правило, яке перевіряє потік даних і шукає в ньому фрагмент SSL-сертифіката, що містить назву центру сертифікації (DigiCert).

```
GNU nano 8.7 local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert ip any any -> any any (msg:"SSL Certificate Detected"; content:"DigiCert", offset 5, depth 50; sid:1000035; rev:1;)
```

Якщо такий фрагмент знаходиться, система генерує сповіщення.

Далі створюю текстовий файл packet.txt, у який записую мережевий пакет у вигляді шістнадцяткових даних. У цей пакет вставляємо фрагмент, що містить частину SSL-сертифіката (тобто рядок із назвою центру сертифікації):

```
(kali@kali)-[~/bkm/lab5]
$ echo "000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00" > packet.txt
echo "000010 00 3c 00 00 40 00 40 06 00 00 c0 a8 01 0a c0 a8" >> packet.txt
echo "000020 01 01 00 50 01 bb 00 00 00 00 00 00 00 00 50 02" >> packet.txt
echo "000030 20 00 00 00 00 00 31 32 33 34 35 44 69 67 69 43" >> packet.txt
echo "000040 65 72 74 2d 47 6c 6f 62 61 6c 2d 52 6f 6f 74" >> packet.txt
```

Це потрібно, щоб надалі перевірити, чи зможе система його виявити.

Далі перетворюю створений текстовий файл packet.txt у формат pcap за допомогою утиліти text2pcap:

```
(kali@kali)-[~/bkm/lab5]
$ text2pcap packet.txt ssl.pcap

Input from: packet.txt
Output to: ssl.pcap (alerts: 0)
Output format: pcapng

Read 1 potential packet, wrote 1 packet (360 bytes including overhead).
```

Це для того, щоб система аналізу трафіку могла обробити цей пакет як звичайний мережевий трафік.

Далі запускаю систему аналізу мережевого трафіку Snort із правилом та подаю їй на вхід створений файл ssl.pcap:

```
(kali@kali)-[~/bkm/lab5]
$ sudo snort -R /etc/snort/rules/local.rules -r ssl.pcap -A alert_fast -k none

o")~ Snort++ 3.11.1.0

Loading rule args: /etc/snort/rules/local.rules
Loading /etc/snort/rules/local.rules:
Finished /etc/snort/rules/local.rules:
Finished rule args: /etc/snort/snort.lua /etc/snort/rules/local.rules ping_lab
```

У результаті аналізу Snort знаходить у пакеті заданий фрагмент SSL-сертифіката та генерує сповіщення “SSL Certificate Detected”:

```
++ [0] ssl.pcap
03/18-12:57:43.000001 [**] [1:1000035:1] "SSL Certificate Detected" [**] [Priority: 0] {TCP} 192.168.1.10:80 → 192.168.1.1:443
-- [0] ssl.pcap
```

Це підтверджує, що правило працює правильно і може виявляти потрібні сертифікати у мережевому трафіку.

Висновки: У ході виконання лабораторної роботи я налаштувала систему виявлення мережових атак Snort та перевірила її роботу на різних типах мережевого трафіку. Я навчилася створювати власні правила (сигнатури) для виявлення конкретних подій у мережі, зокрема ICMP-запитів, HTTP-автентифікації, Cookie, передачі файлів, SNMP-запитів та SSL-сертифікатів. У процесі роботи я також на практиці ознайомила з інструментами аналізу трафіку, такими як tcpdump та text2pcap, і навчилася формувати тестові мережеві пакети для перевірки правил.